

Apply filters to SQL queries

Project description

My organization is working to strengthen system security. As part of this effort, my responsibilities include investigating potential security incidents and ensuring employee computers are updated with the latest security patches. The following sections detail how I used SQL query filters to perform security-related tasks across two key tables:

`log_in_attempts`, which tracks user login activity, and `employees`, which contains information about staff and their assigned machines.

Retrieve after hours failed login attempts

A potential security incident was identified outside of business hours. To investigate, I needed to retrieve all failed login attempts that occurred after 18:00.

```
ariaDB [organization]> clear
ariaDB [organization]> select *
  -> from log_in_attempts
  -> where login_time > '18:00' and success = 0;

+-----+-----+-----+-----+-----+-----+
| event_id | username | login_date | login_time | country | ip_address |
+-----+-----+-----+-----+-----+-----+
| 2 | apatel | 2022-05-10 | 20:27:27 | CAN | 192.168.205.1 |
| 18 | pwashing | 2022-05-11 | 19:28:50 | US | 192.168.66.14 |
| 20 | tshah | 2022-05-12 | 18:56:36 | MEXICO | 192.168.109.5 |
| 28 | aestrada | 2022-05-09 | 19:28:12 | MEXICO | 192.168.27.57 |
| 34 | drosas | 2022-05-11 | 21:02:04 | US | 192.168.45.93 |
| 42 | cgriffin | 2022-05-09 | 23:04:05 | US | 192.168.4.157 |
| 52 | cjackson | 2022-05-10 | 22:07:07 | CAN | 192.168.58.57 |
```

Hours failed login attempts

Your team is investigating failed login attempts that were made after business hours. You want to retrieve this information from the login activity. You'll identify all unsuccessful attempts after 18:00.

The `login_time` column in the `log_in_attempts` table contains information on when login attempts were made. Office hours end at '18:00'.

The `success` column in the `log_in_attempts` table contains values of `TRUE` or `FALSE` to indicate whether the login was successful. MySQL stores Boolean values as 1 for `TRUE`, and 0 for `FALSE`. This means that `TRUE` is represented as 1, and `FALSE` represented as 0 in the `success` column.

- Use the `AND` operator to retrieve the failed login attempts that occurred after business hours. Replace the `X` and `Y` with the correct values to filter for the records you need:

The query below filters the `log_in_attempts` table using a `WHERE` clause combined with an `AND` operator. The first condition, `login_time > '18:00'`, isolates attempts made after business hours. The second condition, `success = FALSE`, narrows the results to only unsuccessful logins. Together, these filters return every failed login attempt that occurred after 18:00.

Retrieve login attempts on specific dates

A suspicious event was flagged on 2022-05-09. To support the investigation, I needed to pull all login activity from that date and the day prior, 2022-05-08.

The query filters the `log_in_attempts` table using a `WHERE` clause with an `OR` operator. The first condition, `login_date = '2022-05-09'`, captures logins on the day of the incident. The second condition, `login_date = '2022-05-08'`, captures logins from the previous day. Using `OR` ensures that records matching either date are included in the results.

```
MariaDB [organization]> select *
-> from log_in_attempts
-> where login_date = '2022-05-08' or login_date = '2022-5-09';
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near 'login_date = '2022-05-08' or login_date = '2022-5-09'' at line 4
MariaDB [organization]> select *
-> from log_in_attempts
-> where login_date = '2022-05-08' or login_date = '2022-05-09';
```

want to retrieve all login attempts that occurred on this day and the day before ('2022-05-08').

The `login_date` column in the `log_in_attempts` table contains information on the dates when login attempts were made.

- Use the `OR` operator to retrieve the failed login attempts on the specified days. Replace the `X` and `Y` with the correct values to filter for the records you need:

The query filters the `log_in_attempts` table using a `WHERE` clause with an `OR` operator. The first condition, `login_date = '2022-05-09'`, captures logins on the day of the incident. The second condition, `login_date = '2022-05-08'`, captures logins from the previous day. Using `OR` ensures that records matching either date are included in the results.

Retrieve login attempts outside of Mexico

Following a review of login activity, I identified a concern with attempts originating outside of Mexico. All such attempts required further investigation.

```
+-----+
rows in set (0.001 sec)

MariaDB [organization]> select *
-> from log_in_attempts
-> where not country like 'mex%';
+-----+
event_id | username | login_date | login_time | country | ip_address
+-----+
1 | irafael | 2022-05-09 | 04:56:27 | CAN | 192.168.243.1
```

Now, your team is investigating logins that did not originate in Mexico, and you need to find this information. Note that the country field includes entries with 'MEX' and 'MEXICO'. You should use the `NOT` and `LIKE` operators and the matching pattern 'MEX%'.

- Run the following SQL query to retrieve login attempts that did not originate in Mexico. Replace `X` with the correct operator and `Y` with the correct pattern to filter for the information you need:

The query filters the `log_in_attempts` table using a `WHERE` clause with the `NOT` operator. To account for the fact that Mexico appears in the dataset as both `MEX` and `MEXICO`, I used `LIKE 'MEX%'` as the pattern. The `%` wildcard matches any sequence of characters following `MEX`, ensuring both representations are excluded. The result set contains only login attempts from countries other than Mexico.

Retrieve employees in Marketing

To roll out a computer update for employees in the Marketing department, I needed to identify which machines to target, specifically those assigned to staff in the East building.

```
mariaDB [organization]> select *
-> from employees
-> where department = 'marketing' and office like 'east%';
```

employee_id	device_id	username	department	office
1000	a320b137c219	elarson	Marketing	East-170
1052	a192b174c940	jdarosa	Marketing	East-195
1075	x573y883z772	fbautist	Marketing	East-267
1088	k865l965m233	rgosh	Marketing	East-157
1103	NULL	randerss	Marketing	East-460
1156	a184b775c707	dellery	Marketing	East-417
1163	h679i515j339	cwilliam	Marketing	East-216

Your team is updating employee machines, and you need to obtain the information about employees in the 'Marketing' department who are located in all offices in the East building (such as 'East-170' or 'East-320').

- Write a SQL query to retrieve this information from the `employees` table. Select all columns and include filters on the `department` and `office` columns to return only the needed records.

Note: You'll need to use the `AND` and `LIKE` operators to satisfy both of these criteria.

The query filters the `employees` table using a `WHERE` clause with an `AND` operator. The first condition, `department = 'Marketing'`, limits results to Marketing staff. The second condition, `office LIKE 'East%'`, matches all East building offices regardless of the specific office number. Both conditions must be true for a record to be returned, ensuring only Marketing employees in the East building are included.

Retrieve employees in Finance or Sales

A separate security update was required for employees in the Finance and Sales departments. I needed to retrieve records for staff in either of these two departments.

```
mariaDB [organization]> select *
-> from employees
-> where department = 'sales' or department = 'finance';
```

employee_id	device_id	username	department	office
1003	d394e816f943	sgilmore	Finance	South-153
1007	h174i497j413	wjaffrey	Finance	North-406
1008	i858j583k571	abernard	Finance	South-170
1009	NULL	lrodriqu	Sales	South-134
1010	k242l212m542	jlansky	Finance	South-109
1011	l748m120n401	drosas	Sales	South-292
1015	p611q262r945	jsoto	Finance	North-271
1017	r550s824t230	jclark	Finance	North-188

as the column in both conditions.

What is the username of the first employee in the Sales department returned by the query?

- ☐ bisles
- ☐ tbarnes



✓ lrodriqu

The query filters the **employees** table using a **WHERE** clause with an **OR** operator. The first condition, **department = 'Finance'**, returns Finance employees. The second condition, **department = 'Sales'**, returns Sales employees. The **OR** operator ensures that employees from either department are included, rather than requiring both conditions to be met simultaneously

Retrieve all employees not in IT

A final security update needed to be applied to all employees outside of the Information Technology department. I first had to identify the relevant employee records.

```
MariaDB [organization]> select *
-> from employees
-> where not department = 'information technology';
```

employee_id	device_id	username	department	office
1000	a320b137c219	el Larson	Marketing	East-170
1001	b239c825d303	bmoreno	Marketing	Central-276
1002	c116d593e558	tshah	Human Resources	North-434
1003	d394e816f943	sgilmore	Finance	South-153
1004	e218f877g788	eraab	Human Resources	South-127
1005	f551g340h864	gesparza	Human Resources	South-366
1007	h174i497j413	wjaffrey	Finance	North-406
1008	i858j583k571	abernard	Finance	South-170
1009	NULL	lrodrigu	Sales	South-134
1010	k242l212m542	jlansky	Finance	South-109

Task 6. Retrieve all employees not in IT

5/6

Your team needs to make one more update. This update was already made to employee computers in the Information Technology department. The team needs information about employees who are not in that department. You should use the **NOT** operator to identify these employees.

- Write a SQL query to retrieve records for employees who are not in the 'Information Technology' department.

How many employees are not in the Information Technology

The query filters the **employees** table using a **WHERE** clause with the **NOT** operator applied to the condition **department = 'Information Technology'**. This returns every employee whose department is anything other than IT, providing the full list of machines that required the update.

Summary

To complete these security tasks, I applied targeted filters to SQL queries pulling from two tables: **log_in_attempts** and **employees**. I used **AND** to combine multiple required conditions, **OR** to capture records meeting at least one of several conditions, and **NOT** to exclude specific groups from results. Where data formatting varied, I used the **LIKE** operator alongside the **%** wildcard to match patterns rather than exact values. These techniques made it possible to retrieve precise, relevant data efficiently at each stage of the investigation.

